



Constraint based generation with Diffusion Models using MCMC Sampling

Under the Guidance of Prof. Sunita Sarawagi

Team members-

Ashutosh Sharma (190100027)

Piyush Sawarkar (22M0772)

Saiful Haq (214057001)

Sakharam Gawade (213050027)



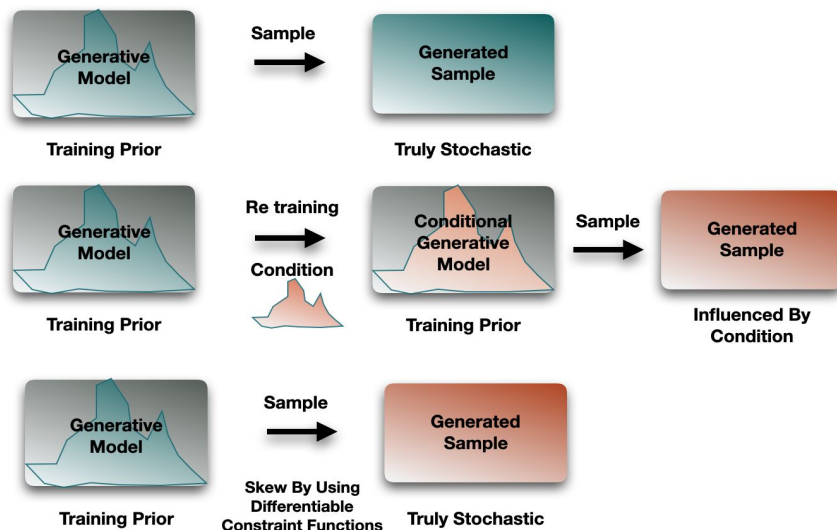


Problem Statement

- 1) There has been many recent works for conditional generation using various generative models like GANs and Diffusion Models, wherein instead of passing a uniform/normally distributed input to the model, they directly input the image/object to condition on as input.
- 2) We aim to solve a generation problem, wherein instead of a an image/object to condition on, we have a constraint we would like the generated object to follow
- 3) Our approach requires a continuous user generated function which scores the input image, giving higher score if constraint is followed and vice-versa, allowing for gradients to backprop.
- 4) We decided to use sampling techniques, because for a model to follow a specific constraint, we do not want to retrain the model from scratch, and want to use a well trained model. Thus requiring no training of the diffusion model.

Motivation

- Generative models like Diffusion models produce truly random samples from the distribution they were trained on.
- To generate a specific desired output, one could use Conditional Generative Models like Conditional Diffusion Models, which require training and render the original model useless.
- The project aims to explore ways to use the trained model to generate desired outputs without retraining it.





Literature Survey

"Arbitrary Style Guidance for Enhanced Diffusion-Based Text-to-Image Generation," proposes a style guidance method for generating high-quality and diverse images using diffusion-based text-to-image generation models. This method can be guided by a reference image to specify fine-grained style features such as color distributions and brush strokes without requiring a separate style transfer model.

"UNIT-DDPM: Unpaired Image Translation with Denoising Diffusion Probabilistic Models," presents a new method for unpaired image-to-image translation using DDPM. This method achieves state-of-the-art performance without adversarial training and generates high-quality images using a denoising Markov Chain Monte Carlo approach.

"Constrained Generative Adversarial Networks for Interactive Image Generation," proposes a GAN framework that allows humans to be "in-the-loop" of the image generation process. This method enables interactive control over image generation without sacrificing image quality by iteratively accepting relative constraints from users and using them to constrain the output of the GAN with respect to an underlying semantic space.

"RePaint: Inpainting using Denoising Diffusion Probabilistic Models," introduces an inpainting approach based on a DDPM that generates high-quality and diverse output images for any inpainting form. This method outperforms state-of-the-art autoregressive and GAN approaches by employing a pretrained unconditional DDPM as the generative prior.

More control for free! Image synthesis with semantic diffusion guidance

- **Conference:** WACV 2023
- **Motivation:**
 - Fine grained controlled over image generation is important for assisting users with art creation and design.
 - Existing text-conditioned synthesis methods require image-caption pairs for model training and do not generalize well to datasets without textual annotations.
 - Existing image-conditioned synthesis methods only transfer the style of reference image to the target image, or these are restricted to domains with well-defined structure (like faces).
- **Methodology:**
 - Input : image/language/both , also called as guidance signal
 - Output: image
 - Method:
 - A guidance function guides the sampling process of an unconditional diffusion model.
 - No need of retraining the diffusion model.
 - Language guidance is based on image to text matching score predicted by CLIP.
 - Authors also propose a self-supervise scheme to finetune the CLIP encoder without textual annotations from which we can obtain the guidance model.
 - Image encoder needs to be trained on noisy images at different time steps.
 - Image guidance
 - Dataset: FFHQ, LSUN

More control for free! Image synthesis with semantic diffusion guidance

$$\begin{aligned}p_{\theta}(x_{t-1}|x_t) &= \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t), \sigma_{\theta}^2(x_t)\mathbf{I}) \\p_{\theta, \phi}(x_{t-1}|x_t, y) &= Z p_{\theta}(x_{t-1}|x_t) p_{\phi}(y|x_{t-1}), \\p_{\theta}(x_{t-1}|x_t) p_{\phi}(y|x_{t-1}) &= \mathcal{N}(\mu + \Sigma g, \Sigma), \\ \mu &= \mu_{\theta}, \Sigma = \sigma_{\theta}^2 \mathbf{I}, g = \nabla_{x_{t-1}} \log p_{\phi}(y|x_{t-1})\end{aligned}$$

- Z is a normalizing constant.
- 'y' is guidance signal
- The new distribution after introduction of guidance signal 'y' in the backward process can be approximated as a Gaussian with shifted mean.
- $p_{\phi}(y|x_{t-1})$ is the probability of x_{t-1} belonging to class 'y'
- Language guidance: $F(x_t, l, t) = E'_I(x_t, t) \cdot E_L(l)$,
- Image guidance: $F(x_t, x'_t, t) = E'_I(x_t, t) \cdot E'_I(x'_t, t)$
- Multimodal guidance: $F_{\phi_0}(x_t, y, t) = s_1 F_{\phi_1}(x_t, y, t) + s_2 F_{\phi_2}(x_t, y, t)$.

Algorithm 1: Semantic Diffusion Guidance

Input: guidance y , scaling factor s

Given: diffusion model $(\mu_{\theta}, \sigma_{\theta})$, Guidance function

$F_{\phi}(x_t, y, t)$

$x_T \leftarrow$ sample from $\mathcal{N}(0, \mathbf{I})$

for $t = T, \dots, 1$ **do**

$\mu, \Sigma \leftarrow \mu_{\theta}, \sigma_{\theta}^2 \mathbf{I}$

$x_{t-1} \leftarrow$ sample from

$\mathcal{N}(\mu + s \Sigma \nabla_{x_t} F_{\phi}(x_t, y, t), \Sigma)$

end for

return x_0

Algorithm for Langevin Monte-Carlo

Input: $E_\theta(\mathbf{x})$, Initial sample $\mathbf{x}^1$.

① For $t = 1 \dots M$

$\mathbf{x} = \mathbf{x}^t$

Compute gradient $\nabla_{\mathbf{x}} E_\theta(\mathbf{x})$

Sample a $\xi \sim N(0, I)$

Compute $\mathbf{x}' = \mathbf{x} - \tau \nabla_{\mathbf{x}} E_\theta(\mathbf{x}) + \sqrt{2\tau} \xi$

Compute $T^Q(\mathbf{x} \rightarrow \mathbf{x}') \propto \exp\left(-\frac{1}{4\tau} \|\mathbf{x}' - \mathbf{x} + \tau \nabla_{\mathbf{x}} E_\theta(\mathbf{x})\|^2\right)$

Compute gradient $\nabla_{\mathbf{x}} E_\theta(\mathbf{x}')$

Compute $T^Q(\mathbf{x}' \rightarrow \mathbf{x}) \propto \exp\left(-\frac{1}{4\tau} \|\mathbf{x} - \mathbf{x}' + \tau \nabla_{\mathbf{x}} E_\theta(\mathbf{x}')\|^2\right)$

$A = \min \left\{ 1, \frac{e^{-E_\theta(\mathbf{x}')} T^Q(\mathbf{x}' \rightarrow \mathbf{x})}{e^{-E_\theta(\mathbf{x})} T^Q(\mathbf{x} \rightarrow \mathbf{x}')} \right\}$

Sample a $u \sim U(0, 1)$

$\mathbf{x}^{t+1} = \mathbf{x}'$ if $u > A$, else $\mathbf{x}$

② Return $\mathbf{x}^1, \dots, \mathbf{x}^M$ as samples



First Approach

- 1) In the diffusion.py, for our experiments, we fixed the function to be the one measuring the amount of redness in the image.
- 2) The function is given as $F = \text{Sum}[(\text{Red}) - 0.5 * (\text{Blue} + \text{Green})]$
- 3) We used Langevin Metropolis Hasting Sampling at each denoising step.
- 4) The input to the scoring function for metropolis hasting was set to the image x_t .

Results from First Approach



Results from First Approach





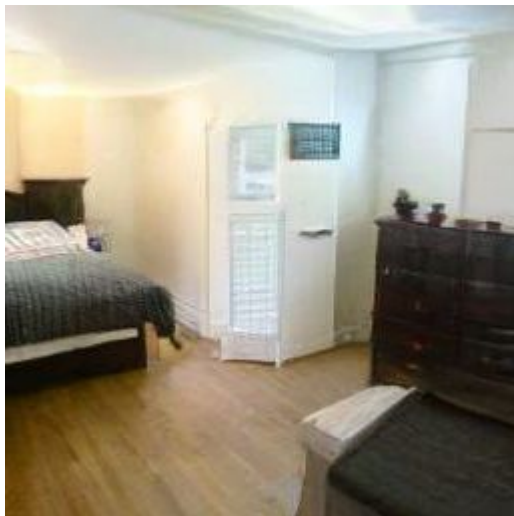
Approach 2A- Use denoised image for taking the MCMC step

- 1) Rather than using MCMC on image x_t , we first get x_0 from x_t by using diffusion denoiser, and use mcmc by calculating score gradient on x_0 calculated and update x_t
- 2) On following above methodology, when we did mcmc updates on each denoising step, running time was around 1 day, so the tau values we checked on gave gaussian noise output.
- 3) But, surprisingly, when we did mcmc steps on subsampled step, let's say every 100 denoising steps, we got some results
- 4) But, on changing the scoring function, to let's say increase greenness of the image, we get the same resulting image, with possibly few minor changes (leading to change in scoring function in decimal places), but visibly the same image

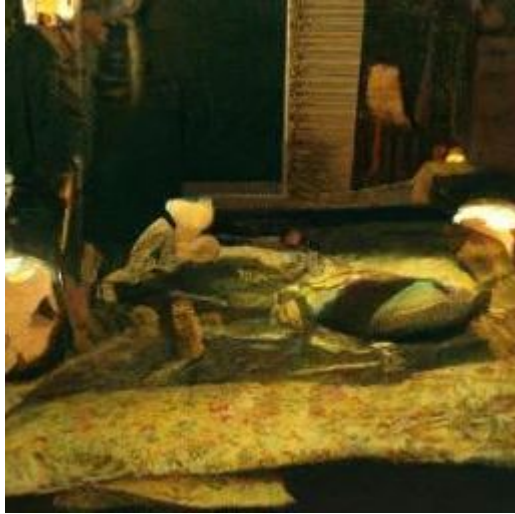
Results 50 -



48



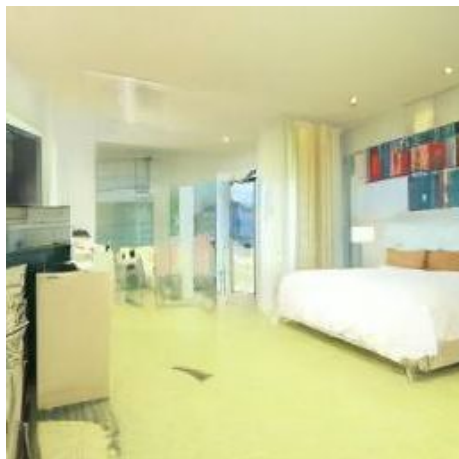
40





Approach 2B- Use denoised image for taking the MCMC step Along with Gaussian Blur

Rather than going for the denoised version x_0 from x_T , we could generate the denoised image from the x_T itself and proceed with the routine MCMC procedure in the reverse diffusion step.







Analysis for approaches 1 and 2

- 1) Approach 1 - On some range of random seeds, the output image changes visibly, but for most of the random seeds, there is no visible change (although the score of image increases in 3rd decimal place).
- 2) Approach 2b - On using gaussian blur for getting denoised image for mcmc update, similar observations were there as approach 1, partly due to the fact that gaussian blur cannot retrieve underlying image from a very noisy image in the initial steps of the diffusion process.
- 3) Approach 2a - On using denoising module of the diffusion model for getting the correct underlying image for mcmc update, surprisingly on doing mcmc update at each step led to bad performance (image is totally noisy, no underlying image visible), but on taking mcmc step every 100 iterations, image changes to give better score (changes in 2nd decimal place, score maximum is 1)

On fixing a random seed, the generated image from 2a model does not change, on changing the function from increasing redness to blue/green. This observation leads us to believe that maybe on fixing the random seed, the input image fixes the output distribution such that very few modes remain, and mcmc update, although tries to bring the image closer to the constraint, the remaining denoising steps bring it back closer to the mode of output distribution.

Third Approach

(Guidance signal on the predicted mean with a feedback loop)

Goal: We want to control the colour of the objects (Red, Blue, Green) in the image generated using reverse process in Diffusion.

Motivation: Existing approaches use guidance signal on the predicted mean to control image generation, however the guidance function need to be trained on the Noisy data. We want introduce a method that can generate target images without the need of model training.[5]

Method:

For i in range(1000, 1):

Mean = predicted using X, where X is the denoised image at the previous step.

Mean_conditioned = 0

If i <= N:

Xmodify = X[if channel != target channel, then pixel = 0]

Mean_conditioned = predicted using Xmodify

Mean = Mean - scaling_factor x (Mean - Mean_conditioned)

X1 is the output after reverse diffusion process

$$\hat{\mu}_{\theta}(x_t, t) = \mu_{\theta}(x_t, t) - s \cdot \Sigma_{\theta}(x_t, t) \nabla_{x_t} |f(x_t) - f(y)|$$

For i in range(1000, 1):

Mean = predicted using X, where X is the denoised image at the previous step.

Mean_conditioned = predicted using X1

Mean = Mean - scaling_factor x (Mean - Mean_conditioned)

$$\hat{\mu}_{\theta}(x_t, t) = \mu_{\theta}(x_t, t) - s \cdot \Sigma_{\theta}(x_t, t) \nabla_{x_t} |f(x_t) - f(y)|$$

Here, N and scaling factor are hyper-parameters.

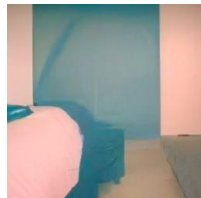
Third Approach

(Guidance signal on the predicted mean without feedback)

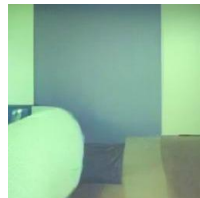
Results:



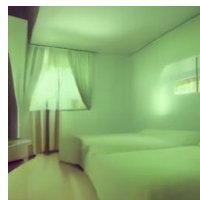
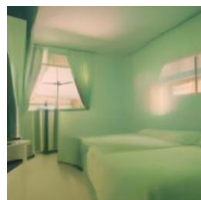
Original Image



Red Colour



Green Colour

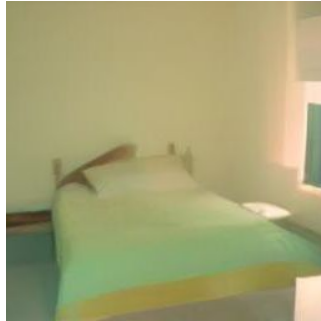


Observation:

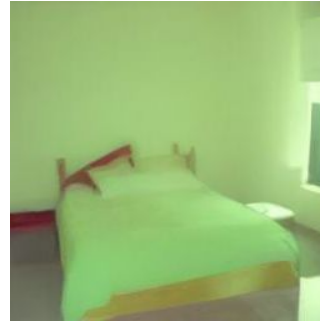
Colour is changed, however few image attributes (Painting at the top left) are missing in the output image.

Third Approach (Guidance signal on the predicted mean with feedback)

Results:



Red Colour Output



Green Colour Output

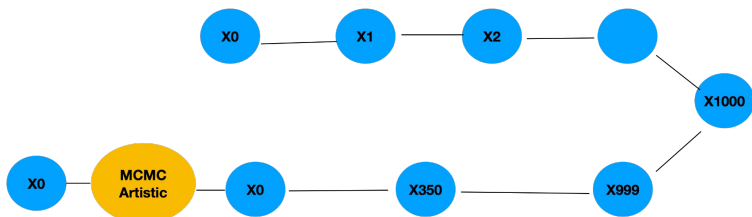
Observation:

Colour is changed, image attributes (bed and window) remain constant in both images.

Fourth Approach

Artistic applied after all steps

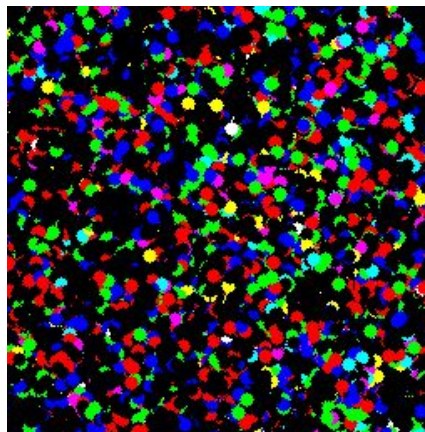
Results



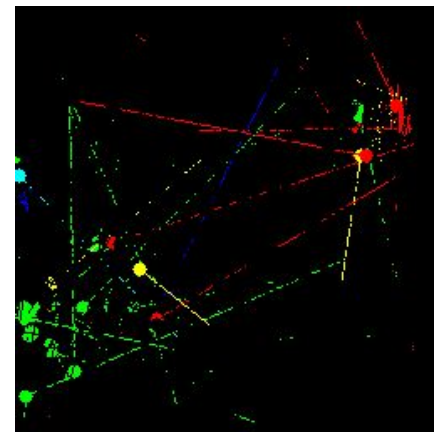
Fourth Approach

Artistic applied at each 100th step

Results:



After 100 Steps



After 1000 steps

References



- [1] Ho, J., Jain, A. and Abbeel, P., 2020. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33, pp.6840-6851.
- [2] Sasaki, H., Willcocks, C.G. and Breckon, T.P., 2021. Unit-ddpm: Unpaired image translation with denoising diffusion probabilistic models. *arXiv preprint arXiv:2104.05358*.
- [3] Heim, E., 2019. Constrained generative adversarial networks for interactive image generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 10753-10761).
- [4] Lugmayr, A., Danelljan, M., Romero, A., Yu, F., Timofte, R. and Van Gool, L., 2022. Repaint: Inpainting using denoising diffusion probabilistic models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 11461-11471).
- [5] Pan, Z., Zhou, X. and Tian, H., 2023. Arbitrary Style Guidance for Enhanced Diffusion-Based Text-to-Image Generation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision* (pp. 4461-4471).
- [6] Liu, X., Park, D.H., Azadi, S., Zhang, G., Chopikyan, A., Hu, Y., Shi, H., Rohrbach, A. and Darrell, T., 2023. More control for free! image synthesis with semantic diffusion guidance. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision* (pp. 289-299).
- [7] Liu, X., Park, D.H., Azadi, S., Zhang, G., Chopikyan, A., Hu, Y., Shi, H., Rohrbach, A. and Darrell, T., 2023. More control for free! image synthesis with semantic diffusion guidance. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision* (pp. 289-299).
- [8] Singh, V., Jandial, S., Chopra, A., Ramesh, S., Krishnamurthy, B. and Balasubramanian, V.N., 2022. On Conditioning the Input Noise for Controlled Image Generation with Diffusion Models. *arXiv preprint arXiv:2205.03859*.
- [9] Choi, J., Kim, S., Jeong, Y., Gwon, Y. and Yoon, S., 2021. Ilvr: Conditioning method for denoising diffusion probabilistic models. *arXiv preprint arXiv:2108.02938*.
- [10] Bau, D., Andonian, A., Cui, A., Park, Y., Jahanian, A., Oliva, A. and Torralba, A., 2021. Paint by word. *arXiv preprint arXiv:2103.10951*.
- [11] Zhang, L., Ma, H., Zhu, X. and Feng, J., 2023. Preconditioned Score-based Generative Models. *arXiv preprint arXiv:2302.06504*.
- [12] Artistic MCMC: <https://aday651.github.io/posts/2022/02/mcmc-images/>
- [13] DCNN: <https://github.com/cszn/DnCNN>
- [14] Diffusion Implementation: https://github.com/pesser/pytorch_diffusion